

## 論文

J. of The Korean Society for Aeronautical and Space Sciences 44(4), 363-372(2016)

DOI: <http://dx.doi.org/10.5139/JKSAS.2016.44.4.363>

ISSN 1225-1348(print), 2287-6871(online)

## SIGMA 큐브위성의 비행 소프트웨어 개발

이정호, 이성환, 이정규, 이효정, 신재혁, 정선영, 오영석, 진 호\*

## Development of Flight Software for SIGMA CubeSat

Jeongho Lee, Seongwhan Lee, JungKyu Lee, Hyojeong Lee, Jehyuck Shin,

Seonyeong Jeong, YoungSeok Oh and Ho Jin\*

School of Space Research, Kyung Hee University,

## ABSTRACT

SIGMA(Scientific cubesat with Instruments for Global Magnetic field and rAdiation) CubeSat has been developed for magnetic field measurement of the Earth and space radiation measurement at Kyung Hee university. The flight software plays important roles in controlling the satellite and processing the data in the space mission. In this paper, the Flight Software has been implemented to process all the tasks in the one thread without RTOS(Real Time Operating System). This is an effective method not only to concentrate the space mission of CubeSat but also to reduce the overhead of the Flight Software by considering the mission perform procedures and the system control methods.

## 초 록

SIGMA(Scientific cubesat with Instruments for Global Magnetic field and rAdiation) 큐브위성은 근 지구 자기장 세기와 우주 방사선량 측정을 위해 경희대학교에서 개발한 초소형 인공위성이다. 우주 임무 수행을 위해 비행 소프트웨어는 위성을 제어하고 데이터를 처리하는 중요한 역할을 담당한다. 본 연구에서는 SIGMA 큐브위성의 임무 수행을 위해 비행 소프트웨어를 구현한 것으로서 일반적으로 임베디드 시스템에서 사용하는 실시간 운영체제를 사용하지 않고 단일 프로세스 내에서 모든 태스크를 처리하도록 구현하였다. 이는 SIGMA 큐브위성의 임무 수행 절차와 시스템 제어 방법을 고려한 것으로서 소프트웨어의 오버헤드(overhead)를 낮추고 임무 수행에 집중할 수 있는 효과적인 방법이다.

**Key Words :** CubeSat(큐브위성), Flight Software(비행 소프트웨어), SIGMA KHUSAT-3 (시그마 KHUSAT-3)

## 1. 서 론

큐브위성은 가로, 세로, 높이 각각이 10 cm이고, 무게가 최대 1.33 kg인 육면체 모양의 초소

형 인공위성이다. 큐브위성은 1999년 California Polytechnic 주립대학교와 Stanford 대학교의 공동 연구로 시작되었으며 위성 개발 비용과 시간을 줄이기 위한 설계 표준을 제공하고 있다[1].

† Received : January 8, 2016      Revised : February 24, 2016      Accepted : February 25, 2016

\* Corresponding author, E-mail : benho@khu.ac.kr

SIGMA(Scientific cubesat with Instruments for Global Magnetic field and rAdiation) 큐브위성은 큐브위성 설계 표준에 따라 경희대학교에서 개발한 초소형 인공위성이다. 위성의 크기는 가로 10 cm, 세로 10 cm, 높이 34 cm이며 무게가 2.9 kg인 3U 형태이다. Fig. 1은 SIGMA 큐브위성의 모습이다.

SIGMA 큐브위성의 우주 임무는 지구 자기장 세기와 우주 방사선량 측정이며 이를 위해 MAG(Magnetometer)와 TEPC(Tissue Equivalent Proportional Counter)를 탑재하고 있다[2].

탑재체 MAG는 3축 Fluxgate Magnetometer로 미국의 New Hampshire 대학교에서 개발하였다. 센서의 분해능은 20 비트까지 디지털 값으로 표현될 수 있으며 0.1 nT 단위까지 측정 가능하고 측정 범위는  $\pm 42000$  nT 이다.

탑재체 TEPC는 한국천문연구원에서 개발한 것으로 우리 인체가 우주 환경에 노출 되었을 때 얼마나 많은 방사선에 피폭이 되는지 확인하기 위해 인체 조직과 등가(等價)한 물질을 이용하여 우주 방사선의 흡수량을 측정한다[3].

이러한 우주 임무 수행을 위해서는 탑재체뿐만 아니라 위성 전체를 제어하는 탑재 컴퓨터가 필요하다. SIGMA 큐브위성의 탑재 컴퓨터는 상용 탑재 컴퓨터를 사용하지 않고 자체 개발한 것

으로 ARM Cortex-M4 기반의 MCU(Micro Controller Unit)를 사용하였다. Fig. 2는 SIGMA 큐브위성의 탑재 컴퓨터 모습이다.

탑재 컴퓨터를 비롯한 각각의 하드웨어를 제어하고 우주 임무를 수행하기 위한 SIGMA 큐브위성의 비행 소프트웨어는 실시간 운영체제를 사용하지 않고 하나의 프로세스 내에서 모든 명령을 처리하는 방식으로 구현하였다. 이는 SIGMA 큐브위성의 우주 임무를 위한 탑재체 제어에 있어서 반드시 실시간 처리를 요구하지 않으며 생성된 데이터는 따로 가공하지 않고 바로 저장하여 지상국으로 전송하는 매우 단순한 구조이기 때문이다. 또한 위성의 전력 시스템, 자세 제어 시스템, 통신 시스템 등은 독립적인 서브시스템으로 탑재 컴퓨터가 직접 제어하지 않고 10초 간격의 충분한 시간 간격으로 시스템을 모니터링하기 때문에 탑재 컴퓨터에 부담이 적다. 뿐만 아니라 위성 운용상에 있어서도 MAG와 TEPC 두 개의 탑재체를 동시에 운영하지 않고 임무 수행 시간을 각각 1개월씩 순차적으로 실행하여 임무 하나하나에 집중할 수 있도록 하였기 때문이다.

이러한 단일 프로세스 방식의 장점은 프로세스가 하나이기 때문에 멀티 프로세스에서 고려해야 하는 뮤텍스(mutex)나 세마포어(semaphore) 같은 데이터 동기화 처리를 하지 않고도 데이터 처리가 가능하며 태스크 전환에 따른 오버헤드(overhead)가 없어서 전체적으로 소프트웨어 복잡도가 낮아지고 실행 속도가 빨라진다. 또한 SIGMA 큐브위성의 탑재 컴퓨터는 자체 개발하였기 때문에 만약 실시간 운영체제를 포팅하려면 하드웨어에 맞게 커널을 수정하고 검증해야 하는 절차가 필요하기 때문에 이에 따른 개발 기간이 길어질 수밖에 없다. 따라서 SIGMA 큐브위성의 비행 소프트웨어는 시스템 제어를 단순화하고 우주 임무 수행을 순차적으로 집중할 수 있는 효율적인 구조와 위성 전체 개발 기간을 고려하여 단일 프로세스 내에서 모든 기능을 처리하도록 구현하였다.

경희대학교에서는 지난 2013년 지구 자기장 측정과 중성입자 검출 임무를 가지고 CINEMA 큐브위성을 발사하여 위성의 HK(Housekeeping) 데이터를 수신한 바 있다. 이 CINEMA 큐브위성의 비행 소프트웨어는 실시간 운영체제를 사용하지 않고 하나의 프로세스로 명령을 수행하도록 구현하였다. SIGMA 큐브위성의 경우 CINEMA 큐브위성과 비교하여 실시간 운영체제를 사용하지 않는다는 점에서는 같지만 위성 운용과 하드웨어 제어를 위한 설계 패턴은 다르다.

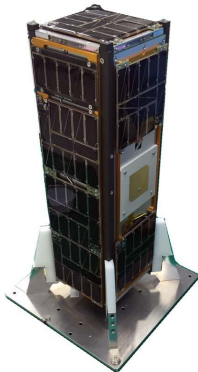


Fig. 1. SIGMA CubeSat



Fig. 2. On board computer

일반적으로 소규모 위성 시스템의 비행 소프트웨어는 중앙제어방식의 설계 패턴(centralized control design pattern)을 사용하는 것이 유용하다. 이 방식은 상태 머신(state machine)을 이용하여 시스템 전반을 제어하는 하나의 컨트롤러(controller)를 사용한다[4]. SIGMA 큐브위성의 비행 소프트웨어는 중앙제어방식의 설계 패턴을 이용하였으며 위성의 각 기능에 따라 작업 단위를 나누고 모듈화(modularity)하여 구현하였다.

본 논문의 구성은 2장 시스템 요구조건, 3장 위성 운용 시나리오, 4장 개발 환경, 5장 비행 소프트웨어 구조, 6장 데이터 처리, 7장 시험 및 검증, 8장 결론으로 구성된다.

## II. 시스템 요구조건

일반적인 인공위성 임무를 설정함에 있어서 기능상, 운용상, 그리고 제약조건에 의한 요구조건들을 고려하여 설계하는 방법이 제시된다[5]. 기능상 요구조건은 위성의 임무 수행을 위한 요구조건이며, 운용상 요구조건은 시스템 운용 방법과 사용자와의 상호작용을 위한 요구조건들이다. 제약조건에 의한 요구조건은 한정된 비용, 일정, 기술 능력 및 주변 환경으로 인한 요구조건들이다[5].

SIGMA 큐브위성에서 요구되는 사항들은 다음과 같다. 기능상 요구조건은 위성 시스템 최초 동작 시 비콘(beacon)을 송신해야하고 위성의 상태정보를 나타내는 HK (housekeeping) 데이터를

주기적으로 수집하며 일정시간 동안 지상국으로부터 명령을 수신하지 못하면 자동으로 탑재체를 실행해야한다.

운용상 요구조건은 대용량 과학 데이터를 통신 패킷 길이에 맞추어 여러 패킷으로 전송해야 하며 MAG 데이터는 시간과 위성 자세 정보를 포함해야 하고 패킷 데이터는 바이너리(binary) 형태로 전송해야한다.

제약 조건에 의한 요구사항은 생산 전력의 한계로 인해 전력 소비가 많은 TEPC와 MAG는 동시에 실행하지 않아야 하며 TEPC는 배터리 전압이 7 V이상일 때만 실행해야한다. Table 1은 주요 요구조건들을 나타낸다.

## III. 위성 운용 시나리오

SIGMA 큐브위성의 임무 기간은 총 3개월이다. 3개월 중 처음 2주간은 HK 데이터를 토대로 위성의 동작상태를 모니터링하고 이후 MAG와 TEPC 탑재체를 각각 1개월씩 실행하여 과학 데이터를 수집한다. Fig. 3은 위성 임무의 전체적인 수행 계획이다.

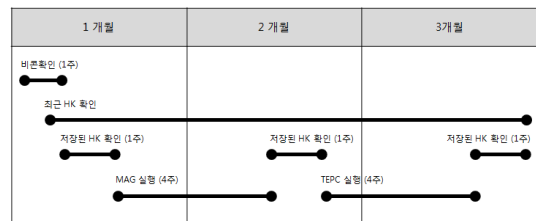


Fig. 3. The schedule of the space mission

Table 1. Requirements of SIGMA CubeSat

| 분류 | 요구조건                                            |
|----|-------------------------------------------------|
| 기능 | 시스템 시작 30분 이후에 안테나 전개                           |
|    | 안테나 전개 이후 비콘 송신                                 |
|    | 위성 상태정보는 10초마다 수집                               |
|    | 모든 데이터는 SD Card에 저장                             |
| 운용 | MAG Boom은 지상국에서 명령을 수신 후 전개                     |
|    | 과학 데이터는 여러 패킷으로 나누어 차례로 전송                      |
|    | MAG 데이터는 시간과 자세정보 포함                            |
|    | SD Card에 저장 공간이 없을 때는 제일 오래된 데이터부터 지우고 저장       |
| 제약 | 통신 패킷은 바이너리 코드로 전송                              |
|    | TEPC는 배터리 전압이 7V 이상에서 실행<br>MAG와 TEPC는 동시 실행 불가 |

Table 2. List of operation mode

| 번호 | 모드           | 설명                                      |
|----|--------------|-----------------------------------------|
| 1  | INIT         | 시스템을 초기 화하고 안테나 전개                      |
| 2  | STABLE       | 비콘 송신하며 지상국 명령 대기                       |
| 3  | NORMAL       | 최근 위성 상태 정보를 송신하며 지상국 명령 대기             |
| 4  | SCIENCE      | 탑재체 MAG와 TEPC 실행                        |
| 5  | SCIENCE AUTO | 일정 시간동안 지상국 명령을 수신하지 못했을 경우 자동으로 탑재체 실행 |
| 6  | ACS          | 위성 자세 제어                                |
| 7  | RECOVERY     | 시스템 오류 발생 시 복구 및 재 초기화                  |

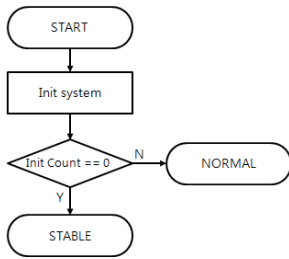


Fig. 4. Flow chart of INIT mode

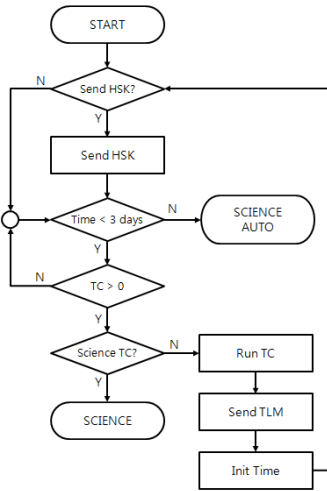


Fig. 5. Flow chart of NORMAL mode

위성의 운용 시나리오는 시스템 요구조건을 바탕으로 Table 2와 같이 7 가지 운용 모드를 가진다. 처음 위성 시스템이 시작되면 INIT 모드가 실행되는데 INIT 모드는 Fig. 4에서 보는 바와 같이 시스템을 초기화하고 안테나를 전개하여 지상국과 통신을 위한 준비를 한다. INIT 모드가 성공적으로 수행되면 STABLE 모드가 실행된다. STABLE 모드는 Fig. 6과 같이 2주 동안 지상으로 비콘(beacon)을 송신하면서 지상국 명령을 수행한다. 그리고 2주 후에는 자동으로 NORMAL 모드가 실행되는데 NORMAL 모드는 Fig. 5와 같이 비콘 대신 최근 HK 데이터를 송신하면서 지상국 명령을 수행한다. 이 때 만약 3일 동안 지상국으로부터 아무런 명령을 수신하지 못하면 자동으로 SCIENCE AUTO모드를 실행하여 과학 데이터를 수집하고 지상으로 전송하며 데이터 전송이 다 끝나면 자동으로 다시 NORMAL 모드로 전환된다. SCIENCE 모드는 Fig. 7과 같이 NORMAL 모드에서 지상국 명령에 의해 실행되며 수집된 데이터는 SD card에 저장 후 다시

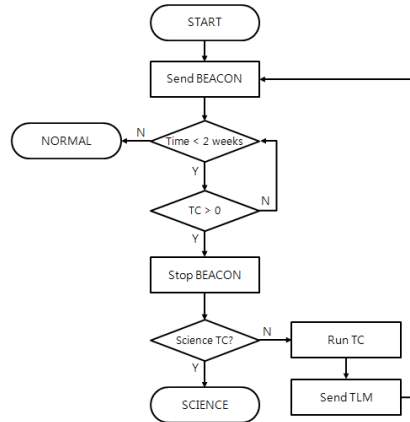


Fig. 6. Flow chart of STABLE mode

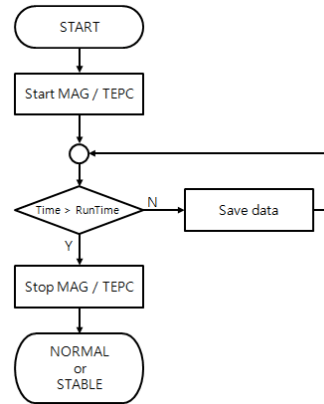


Fig. 7. Flow chart of SCIENCE mode

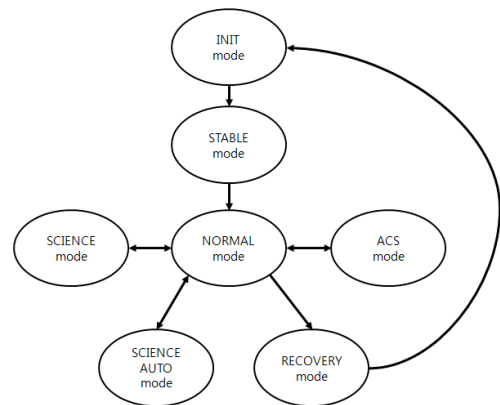


Fig. 8. Transition diagram of the operation mode

NORMAL 모드로 전환된다. Fig. 8은 전체적인 운용 모드의 실행 구조를 나타낸다.

## IV. 개발 환경

SIGMA 큐브위성의 탑재 컴퓨터는 ARM Cortex-M4 기반의 MCU(Micro Controller Unit)를 사용하기 때문에 비행 소프트웨어는 ARM용 실행파일로 컴파일 되어야 한다. 반면 개발에 사용된 호스트(host) PC는 인텔 x86 기반의 CPU와 윈도우 7 운영체제를 사용한다. 따라서 ARM용 크로스 컴파일(cross-compile) 환경을 구축하기 위해 KEIL사에서 제공하는 ARM용 MDK(Microcontroller Development Kit)를 사용하였다. MDK에는 소스 코드 편집을 위한 에디터(editor)와 크로스 컴파일을 위한 ARM용 C/C++ 컴파일러가 포함되어 있다[6].

소스 코드는 C 프로그래밍 언어를 사용하여 작성하였고 SIGMA 큐브위성의 탑재 컴퓨터에 사용된 MCU가 ST사 제품이기 때문에 ST사에서 제공하는 라이브러리를 함께 사용하였다.

컴파일된 실행 파일은 ST-Link USB Dongle 장비로 호스트 PC와 위성 탑재 컴퓨터를 연결하고 ST-Link 유틸리티 프로그램을 이용하여 다운로드 하였다. Fig. 9는 실행파일을 탑재 컴퓨터에 다운로드하는 모습이다.

## V. 비행 소프트웨어 구조

SIGMA 큐브위성의 비행 소프트웨어는 실시간 운영체제를 사용하지 않고 하나의 프로세스에서

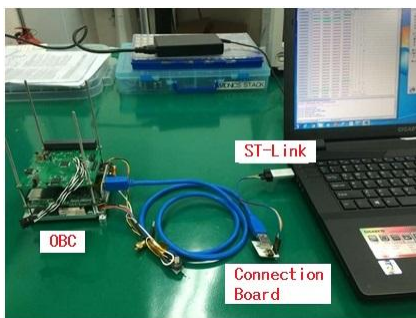


Fig. 9. Download the program to the OBC

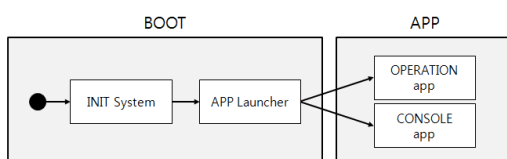


Fig. 10. The structure of flight software

위성의 모든 기능을 처리할 수 있도록 구현하였다. 이를 위해 먼저 위성의 각 기능들을 분류하여 독립적으로 실행할 수 있도록 모듈화(modularity)하였다. 모듈화는 코드의 재사용성을 높이고 소프트웨어의 구조를 단순화 할 수 있으며 수정이 쉬운 장점을 가지고 있다[7].

모듈화된 기능들을 계층화하여 각 모듈간의 상호동작이 가능하도록 하였고 위성운용 시나리오에 따라 7개의 운용 모드들을 각각의 태스크로 구분하여 각 모듈을 제어할 수 있도록 구현하였다.

비행 소프트웨어의 구조는 시스템 초기화를 위한 BOOT 부분과 임무 수행을 위해 응용 프로그램이 실행되는 APP(application) 부분으로 나누어 구현하였다. APP 부분은 OPERATION 또는 CONSOLE 응용 프로그램을 선택적으로 실행할 수 있다. Fig. 10은 비행 소프트웨어 구조를 나타낸다.

### 5.1 BOOT 구조

BOOT 부분은 INIT System과 APP Launcher로 구성된다. INIT System은 다른 서브시스템과의 통신을 위한 UART, I<sup>2</sup>C, SPI 인터페이스를 초기화 하고 System Tick, RTC(Real Time Clock), Timer를 차례로 초기화 한다.

APP Launcher는 사용자 선택에 따라 OPERATION 또는 CONSOLE 응용 프로그램을 선택적으로 실행하도록 한다. 만약 5분 동안 사용자의 입력이 없으면 자동으로 OPERATION 응용 프로그램을 실행한다.

### 5.2 APP 구조

APP 부분은 OPERATION 응용 프로그램과 CONSOLE 응용 프로그램으로 나누어지는데 APP Launcher에 의해 OPERATION 또는 CONSOLE 응용 프로그램이 동작하도록 구현하였다.

OPERATION 응용 프로그램은 위성 운용 시나리오에 따라 나누어진 7개 모드를 각각의 태스크로 구분하여 태스크 매니저(task manager)가 필요에 따라 적절한 태스크를 선택하여 실행할 수 있도록 하였다. 이로써 비행 소프트웨어 내부적으로 중복되는 기능들을 하나로 통합하고 단순화 하여 효과적인 명령 수행이 가능하도록 하였다. Fig. 11은 OPERATION 응용 프로그램의 동작 구조를 나타낸 것으로 태스크 매니저가 운용 시나리오에 따라 태스크를 실행한다. 각 태스크 내에서는 상태 매니저(state manager)가 무한 반

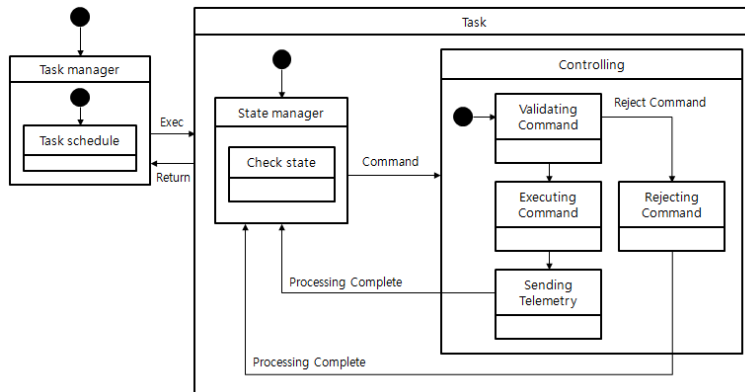


Fig. 11. The structure of OPERATION app

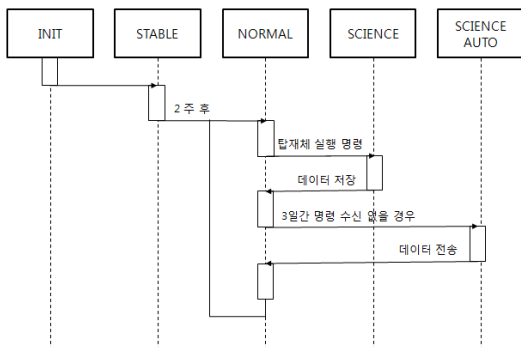


Fig. 12. Sequence diagram of the tasks

복되면서 태스크의 상태 값을 검사하여 각 상태에 따라 지상국 명령을 실행하거나 자동화된 명령들을 수행하게 된다. 명령을 수행할 때는 먼저 명령 코드와 파라미터 값을 검사하고 만약 명령이 올바르지 않을 경우 바로 상태 매니저로 리턴 한다. 정상적인 경우에는 명령을 수행하고 결과 값을 지상으로 전송한다.

각각의 태스크는 Fig. 12와 같은 순서로 실행된다. 먼저 INIT 태스크가 실행된 후 STABLE 태스크가 2주 동안 실행되며 각 중 명령을 수행한다. 이 후 NORMAL 태스크가 실행되고 만약 지상국으로부터 탐재체 실행 명령을 수신하면 SCIENCE 태스크를 실행한다. 그리고 3일 동안 어떠한 명령도 수신하지 못할 경우 자동으로 SCIENCE AUTO 태스크가 실행되고 이러한 일련의 과정이 반복된다.

CONSOLE 응용 프로그램은 사용자가 직접 명령어를 입력하여 위성을 제어하는 프로그램으로 지상에서 위성을 수동으로 제어함으로써 각각의 기능 동작을 시험하고 위성의 상태를 모니터링



Fig. 13. The screen of CONSOLE app

할 수 있도록 하였다. 이를 실행하기 위해서는 사용자 PC와 위성의 탑재 컴퓨터가 시리얼 케이블로 연결되어 있어야 하며 사용자는 PC용 시리얼 프로그램을 이용하여 명령어를 전송하고 그 결과를 확인할 수 있다. Fig. 13은 PC에서 시리얼 프로그램을 이용해 명령을 수행하는 화면이다.

## VI. 데이터 처리

SIGMA 큐브위성에서 처리하는 데이터의 종류는 위성 상태 정보를 담고 있는 HK 데이터, MAG 탑재체로부터 얻어진 자기장 측정 데이터, TEPC 탑재체로부터 얻어진 방사선 흡수량 데이터, 지상국으로부터 실행할 명령 코드를 담고 있는 Telecommand 데이터, 명령 수행 결과를 지상국으로 전송할 Telemetry 데이터가 있다. 여기서 HK, MAG, TEPC 데이터는 SD card에 저장



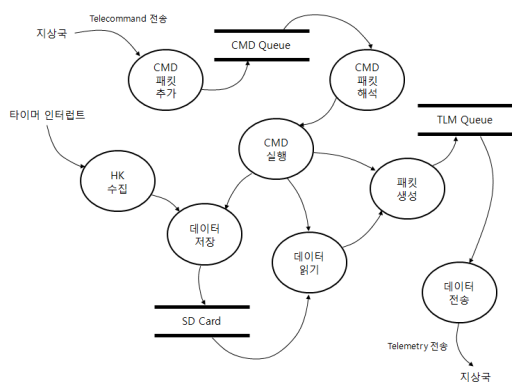


Fig. 14. Data flow diagram

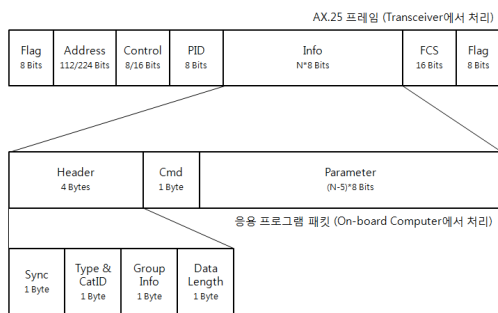


Fig. 15. The structure of the packet data

되어 관리되고 Telecommand와 Telemetry는 메모리에 저장되어 관리된다.

전체적인 데이터의 흐름은 Telecommand를 수신하면 패킷을 CMD(command) 큐(queue)에 추가하고 각 명령을 해석하여 실행한다. 명령 실행 후 생성된 데이터는 SD card에 저장하거나 바로 패킷 형태로 변환하여 TLM(telemetry) 큐에 저장한다. 이후 TLM 큐의 데이터를 지상국으로 전송한다. 이와 동시에 타이머 인터럽트에 의해 10 초마다 HK 수집을 실행하고 HK 데이터를 SD card에 저장한다. Fig. 14는 전체적인 데이터 흐름을 나타낸다.

## 6.1 통신 데이터 처리

SIGMA 큐브위성의 통신은 AX.25 프로토콜을 이용하여 지상국과 통신한다. AX.25 프로토콜은 두 단말기(terminal) 사이의 데이터 교환을 보다 안정적이고 높은 신뢰성을 확보하기 위해 아마추어 무선 단체에서 고안한 것으로 프레임은 Flag, Address, Control, PID, Info, FCS 필드로 구성된 다[8].

AX.25 프레임 데이터 처리는 트랜시버

Table 3. List of category ID

| Subsystem                  | Category ID |
|----------------------------|-------------|
| On-board Computer          | 0x01        |
| Electronic Power System    | 0x02        |
| Battery                    | 0x03        |
| Instrument Interface Board | 0x04        |
| Transceiver                | 0x05        |
| ACS                        | 0x06        |
| Antenna                    | 0x07        |
| SD Card                    | 0x08        |
| MAG                        | 0x09        |
| TEPC                       | 0x0A        |
| Common                     | 0x0F        |

(transceiver)에서 담당한다. 따라서 비행 소프트웨어에서는 AX.25 프레임 내의 Info 필드에 저장되는 응용 프로그램용 패킷만을 처리한다.

응용 프로그램용 패킷은 헤더(header) 4 바이트, 명령 코드 1 바이트, 그리고 파라미터(parameter)로 구성하였다. 파라미터의 길이는 가변적인데 TC(tele-command)의 경우 최대 195 바이트이고, TLM(telemetry)의 경우 최대 230 바이트이다. Fig. 15는 패킷의 구조를 나타낸다.

패킷의 헤더는 패킷 시작을 알리는 Sync 바이트(0xCC)와 패킷 타입, 카테고리(category), 그룹 패킷 여부와 패킷 번호, 그리고 명령 코드와 파라미터 길이로 구성하였다. 패킷 타입은 TC와 TLM을 구분하고 카테고리는 패킷을 처리할 서브시스템을 나타낸다. Table 3은 각 서브시스템 카테고리 ID를 정리한 것이다.

TC와 TLM 데이터는 환형 큐(circular queue) 데이터 구조를 이용하여 관리하도록 구현하였다. 환형 큐 데이터 구조는 1차원적인 배열 형태의 큐를 원형으로 구성한 것이다. 이는 데이터를 저장할 때 저장 공간이 배열의 끝에 도달하면 다시 배열의 처음부터 저장하는 방식이다[9]. Fig. 16은 전체적인 패킷 처리 구조를 내는데 지상으로 부터 받은 모든 TC 패킷은 command 환형 큐에 저장된다. 이후 각 패킷은 그룹 패킷인지를 검사하여 그룹 패킷이면 하나로 합쳐져 실행된다. 그리고 실행 결과 데이터 길이가 한 프레임을 넘을 경우 그룹 패킷으로 나누어 전송한다. 예를 들어 지상에서 탑재체 MAG 데이터 한 블록을 내려 받으려면 먼저 SD Card 읽기 명령과 MAG 데이터 블록 번호를 하나의 패킷으로 구성하여 위성으로 전송한다. 위성은 수신된 해당 패킷을 command 환형 큐에 추가하고 해당 명령을 수행

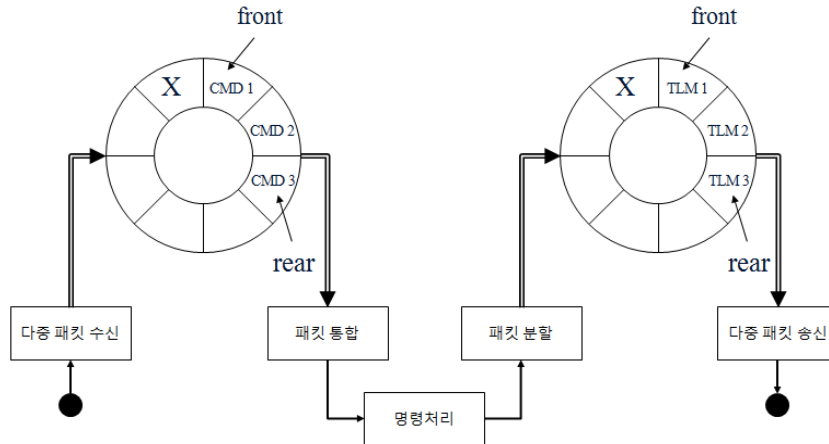


Fig. 16. Communication packet processing

한다. 이때 해당 명령은 하나의 패킷으로 구성되기 때문에 패킷 통합 과정은 필요하지 않다. 패킷 통합 과정은 ACS의 TLE(Two Line Elements) 데이터를 업데이트하는 명령과 같이 여러 개의 패킷으로 구성될 경우에 실행된다. 명령 실행 이후 SD Card로부터 읽은 MAG 데이터는 한 지상으로 전송하기 위해 패킷 형태로 변환되는데 이때 MAG 데이터 한 블록은 512 바이트이고 하나의 패킷은 최대 235 바이트이기 때문에 패킷 헤더를 포함하여 모두 3개의 패킷으로 나누어 구성된다. 이렇게 구성된 패킷은 telemetry 환형 큐에 추가되어 지상으로 전송된다.

## 6.2 데이터 저장

10초 마다 수집된 HK 데이터와 과학 데이터는 모두 SD Card에 저장한다. SIGMA 큐브위성의 SD Card는 SPI 방식으로 MCU와 통신한다.

데이터 저장 방식은 HK 데이터와 MAG 및 TEPC 과학 데이터의 생성량을 고려하여 SD Card에 파티션(partition)을 나누고 512 바이트 단위의 블록으로 읽고 쓰도록 하였다. 이 때 블록 접근 방법은 따로 파일 시스템을 사용하지 않기 때문에 일반적인 파일 형태가 아닌 메모리 주소만을 이용한 접근 방법을 사용하였다. Table 4는 SD Card의 파티션 정보이다.

각 파티션에 블록 저장 시 파티션의 마지막 블록까지 모두 저장되어 있는 경우 파티션의 처음부터 다시 저장하는 환형 구조를 갖도록 구현하였다. 이때 파티션의 앞 블록에 있는 데이터를 지상국에서 다운로드 하지 못했다면 데이터 손실이 일어나는 단점이 있다. 그럼에도 불구하고 이러한 방식을 사용한 이유는 이전 데이터보다 최

Table 4. Partition information of SD card

| Block index |           | Data        | Capacity  |
|-------------|-----------|-------------|-----------|
| Start       | End       |             |           |
| 0           | 0         | System info | 512 bytes |
| 1           | 1,000,000 | HK          | ≒ 500 MB  |
| 1,000,001   | 2,800,000 | MAG         | ≒ 900 MB  |
| 2,800,001   | 3,800,000 | TEPC        | ≒ 500 MB  |

근 데이터를 유지해야하는 시스템 요구조건 중 하나이기 때문이다.

## VII. 시험 및 검증

SIGMA 큐브위성의 비행 소프트웨어 검증은 두 가지 방식으로 진행 되었다. 하나는 CONSOLE app을 실행한 상태에서 각 서브시스템들의 동작 기능을 시험하였고 다른 하나는 OPERATION app을 실행한 상태에서 무선 통신 장비를 이용하여 AX.25 프로토콜로 명령을 송신하고 결과 데이터를 수신함으로써 소프트웨어의 동작을 검증하였다.

CONSOLE app을 이용한 검증은 각 서브시스템들의 동작을 위한 드라이버 소프트웨어 기능을 검증하였다. 시험 환경은 먼저 각 서브시스템들을 PC104 커넥터가 서로 연결되어 있는 시험용 Flat 보드에 연결하고 PC와 탑재 컴퓨터를 연결한 다음 각 서브시스템에 명령을 하나씩 전달하여 데이터를 정상적으로 읽어 오는지 확인하였다. Fig. 17은 CONSOLE 시험 환경이다.



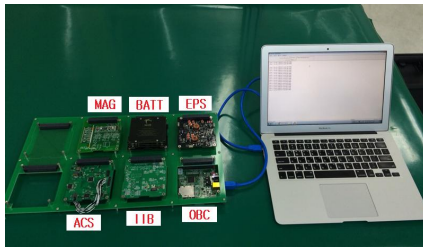


Fig. 17. Test of the CONSOLE app

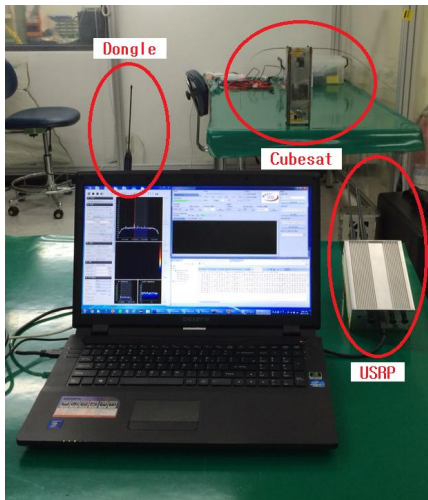


Fig. 18. Test of the OPERATION app

OPERATION app을 이용한 검증은 운용 시나리오에 따른 각 태스크의 동작을 검증하였다. 시험 환경은 상향링크(Uplink) 통신을 위해 USRP(Universal Software Radio Peripheral) 무선 장비를 이용하였고, 하향링크(Downlink) 통신을 위해 FUN Cube용 Dongle 안테나를 사용하였다. 시험은 USRP 장비를 이용하여 명령 코드를 위성으로 전송하고 위성에서 해당 결과 데이터가 Dongle 안테나를 통해 들어오는지 확인하였다. Fig. 18은 무선 장비를 이용한 OPERATION 시험 환경이다.

## VIII. 결 론

SIGMA 큐브위성의 비행 소프트웨어는 위성 및 탑재체 제어와 임무 수행 방식을 고려하여 개발하였다. 비행 소프트웨어의 구조는 중앙 제어 방식의 설계 패턴을 바탕으로 위성의 각 기능들을 모듈화하고 운용 시나리오에 따라 7개의 태스크를 정의하여 구현하였다. 이는 소프트웨어의 구조적 측면과 동작 실행에 있어서 복잡성을 낮

추고 효율을 증대시켜 위성 본연의 임무를 효과적으로 수행할 수 있도록 하였다.

비행 소프트웨어의 BOOT 부분은 시스템을 초기화하고 APP 부분은 OPERATION 또는 CONSOLE 응용 프로그램을 선택적으로 실행할 수 있도록 하였다.

데이터 처리에 있어서 각 통신 패킷 데이터는 환형 큐 구조로 관리할 수 있도록 하였으며 HK 데이터와 과학 데이터는 512 바이트의 블록 단위로 SD Card에 저장하도록 하였다.

비행 소프트웨어 검증은 먼저 CONSOLE 응용 프로그램 실행 상태에서 각 서브시스템에 명령을 수행하여 정상 동작하였음을 확인하였다. 그리고 OPERATION 응용 프로그램 실행 상태에서 지상국의 무선 통신 시스템을 이용하여 HK 데이터, 탑재체 실행 및 과학 데이터를 수신하여 위성의 정상 동작을 확인하였다.

본 소프트웨어는 운영시험을 거쳐 발사를 준비 중이며, 향후 위성운행을 통해 보다 개선된 방향으로 발전할 수 있을 것이다. 예를 들면 무선 통신에서 발생할 수 있는 데이터 손실을 감안하여 보다 효과적인 패킷 처리 방식이나 위성 운영을 위한 명령어 구조 등을 보다 효율적으로 구성할 수 있을 것이다. 본 연구에서 제시한 비행 소프트웨어 구조 및 동작 방법은 SIGMA 큐브위성의 독립적이고 간결한 탑재체 제어와 순차적인 임무 수행 방식의 이유로 효과적인 방법이다.

## References

- 1) Puig-Suari, J., "CubeSat Design Specification Rev. 13," *The CubeSat Program, Cal Poly SLO*, 2014.
- 2) Lee S., Lee J. K., Kum K. et al., "Development of SIGMA(KHUSAT-3) CubeSat Mission" SSC14-P4-2, SmallSat Conference, 2014
- 3) Nam, U. W., Park, W. K., Lee, J. J., et al., "Calibration of TEPC for CubeSat Experiment to Measure Space Radiation," *Journal of astronomy and space sciences*, 2015, pp.145~149.
- 4) Fant, J., Gomaa, H., and Pettit, R., "Architectural Design Patterns for Flight Software," 2011 14th IEEE International Symposium on Object/Component/Service - Oriented Real-Time Distributed Computing Workshops, 2011, pp.97~101.
- 5) Yoon, S. Y, Kim, Y. H, Yoon, J. W, Seon, J. H, Jin, H, Lee, D. H, "Initial Operational

Scenario of TRIO-CINEMA Spacecraft," Journal of the Korean society for aeronautical & space sciences, 2012 , pp.1053~1058.

6) <http://www.keil.com>

7) Hansen, L., and Hansen, J., "Reusable, modular, and scalable flight software," *Aerospace Conference, 2014 IEEE*, 2014, pp.1~7.

8) William A. Beech, Douglas E. Nielsen, and Jack Taylor, "AX.25 Link Access Protocol for Amateur Packet Radio Version 2.2," *Tucson Amateur Packet Radio Corporation*, 1998.

9) Horowitz, E., Sahni, S., and Anderson-Freed, S., *Fundamentals of data structures in C*, Silicon Press, 2007